UNIVERSIDADE DE FORTALEZA

VICE REITORIA DE GRADUAÇÃO

CENTRO DE CIÊNCIAS TECNOLÓGICAS

TECNÓLOGO EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Carlos Alberto Freitas (2422750)

Andrei Araripe (2419091)

DOCUMENTAÇÃO TÉCNICA

NutriFlow



FORTALEZA

2026

Carlos Alberto Freitas (2422750)

Andrei Araripe (2419091)

DOCUMENTAÇÃO TÉCNICA

NutriFlow



Este documento apresenta a documentação técnica do sistema NutriFlow, desenvolvido na componente curricular N392 - Projeto Aplicado Plataformas Web como requisito para obtenção de nota.

Supervisor: **Prof.ª Cynthia Moreira**

FORTALEZA

2026

SUMÁRIO

1. INTRODUÇÃO

1.1 CONTEXTO E JUSTIFICATIVA

1.2 OBJETIVOS

1.3 ESCOPO E DELIMITAÇÃO

2. ENGENHARIA DE REQUISITOS

2.1 REQUISITOS FUNCIONAIS (RFs)

2.2 REQUISITOS NÃO FUNCIONAIS (RNFs)

3. PROJETO E ARQUITETURA DO SOFTWARE

3.1 ARQUITETURA GERAL

3.2 PROJETO DO BANCO DE DADOS

3.3 PROJETO DE API

4. TECNOLOGIAS E FERRAMENTAS

4.1 STACK DE TECNOLOGIAS

4.2 FERRAMENTAS DE DESENVOLVIMENTO

5. IMPLEMENTAÇÃO E RESULTADOS

5.1 TELAS DO SISTEMA

6. AMBIENTE E GUIA DE IMPLANTAÇÃO

6.1 REQUISITOS DO AMBIENTE

6.2 PROCESSO DE IMPLANTAÇÃO

6.3 ACESSO À APLICAÇÃO IMPLANTADA

7. CONCLUSÃO

7.1 TRABALHOS FUTUROS

7.2 GRUPO DE TRABALHO

7.3 LIÇÕES APRENDIDAS

1. INTRODUÇÃO

1.1 CONTEXTO E JUSTIFICATIVA

O NutriFlow responde à fragmentação do acompanhamento nutricional, que frequentemente combina planilhas, mensagens e registros separados. A plataforma centraliza a relação entre pacientes, nutricionistas e administradores, organizando informações de alimentação, evolução e comunicação.

Pacientes e nutricionistas podem manter informações de acompanhamento em planilhas, conversas e anotações distintas. Essa dispersão dificulta a consulta ao histórico, o acompanhamento de metas e a comunicação contínua entre profissional e paciente.

O NutriFlow centraliza perfis, planos alimentares, refeições, medidas, desafios, consultas e chat em uma plataforma web. Seu público inclui pacientes, nutricionistas e administradores; a solução busca tornar o acompanhamento mais organizado, rastreável e acessível.

1.2 OBJETIVOS

Objetivo geral: desenvolver uma plataforma web que organize o acompanhamento nutricional e facilite a comunicação entre pacientes, nutricionistas e administradores.

Objetivos específicos: permitir o registro e a consulta de refeições, peso e medidas; apoiar a criação de planos alimentares e desafios; organizar consultas e comunicação entre os perfis; e oferecer administração de usuários, aprovações e catálogo de alimentos.

Disponibilizar cadastro e autenticação de pacientes e nutricionistas, com confirmação de e-mail e recuperação de senha.

Permitir o registro e acompanhamento de refeições, peso, medidas corporais e progresso de desafios.

Permitir a criação de planos alimentares e consultas pelo nutricionista para pacientes vinculados.

Oferecer comunicação por mensagens de texto e áudio, além de ferramentas administrativas para usuários e alimentos.

1.3 ESCOPO E DELIMITAÇÃO

O escopo contempla as funcionalidades efetivamente implementadas no sistema. As atividades clínicas permanecem sob responsabilidade dos profissionais; a plataforma oferece suporte à organização e ao registro.

Escopo:

Cadastro e autenticação de pacientes e nutricionistas, com perfis e controle de acesso.

Vínculo de pacientes a nutricionistas e consulta a registros de acompanhamento.

Criação de planos alimentares, com alimentos, quantidades e informações nutricionais.

Registro de refeições, peso, medidas corporais, desafios e check-ins.

Chat de texto e áudio, consultas entre paciente e nutricionista, e administração de usuários, aprovações e catálogo de alimentos.

Delimitação (Fora do Escopo):

O sistema não substitui prontuário eletrônico, diagnóstico, prescrição clínica automatizada ou atendimento profissional.

Não há integração com convênios, operadoras de saúde ou sistemas externos de clínicas.

Não estão incluídos pagamentos, faturamento ou emissão de documentos fiscais.

A aplicação não oferece telemedicina ou videochamadas.

Não há aplicativo móvel nativo, notificações automáticas por WhatsApp/e-mail ou integrações externas de mensagens.

2. ENGENHARIA DE REQUISITOS

2.1 REQUISITOS FUNCIONAIS (RFs)

ID	Nome do Requisito	Descrição
RF01	Cadastro e perfis	Permitir o cadastro público de pacientes e nutricionistas com confirmação de e-mail. A criação do perfil administrador não é disponibilizada pelo cadastro público.
RF02	Acesso à conta	Permitir login, logout, confirmação e reenvio de e-mail, recuperação de senha e consulta dos dados do usuário autenticado.
RF03	Perfil e foto	Permitir ao usuário consultar e atualizar os dados do próprio perfil, incluindo nome, telefone e, para o perfil de paciente, informações pessoais; incluir ou trocar a foto de perfil.
RF04	Vínculo paciente–nutricionista	Permitir ao paciente informar o nutricionista responsável e ao nutricionista vincular um paciente. Registrar objetivo, idade e restrições informados no vínculo.
RF05	Aprovação de nutricionistas	Permitir ao administrador consultar solicitações após a confirmação de e-mail e aprovar ou rejeitar o cadastro, registrando o motivo da rejeição. Exigir aprovação para liberar o acesso profissional.
RF06	Administração de usuários	Permitir ao administrador pesquisar e filtrar usuários, atualizar dados, bloquear ou reativar contas e remover usuários.
RF07	Registro de refeições	Permitir ao paciente registrar o tipo, horário, descrição e valores nutricionais das refeições, inclusive fibras e água consumida em mililitros.
RF08	Resumo alimentar	Apresentar ao paciente os totais diários de calorias, macronutrientes, fibras e água, comparações com as metas do plano, histórico alimentar e consumo de calorias por dia da semana.
RF09	Registro e evolução do peso	Permitir registrar pesagens com data e observação e consultar o histórico, gráfico, variação, tendência e média semanal do peso.
RF10	Avaliações antropométricas	Permitir ao nutricionista registrar, consultar, editar e remover avaliações de pacientes vinculados, incluindo peso, altura, percentual de gordura, circunferências, dobras cutâneas e observações.

Requisitos funcionais — continuação.

ID	Nome do Requisito	Descrição
RF11	Acompanhamento de pacientes	Permitir ao nutricionista consultar a lista de pacientes vinculados, seus dados de perfil, objetivos, restrições, pesagens e histórico de medidas corporais.
RF12	Catálogo de alimentos	Permitir ao administrador consultar, cadastrar, remover e reativar alimentos com seus dados nutricionais; permitir ao nutricionista consultar o catálogo.
RF13	Planos alimentares	Permitir ao nutricionista criar planos para pacientes vinculados, selecionar alimentos, quantidades e refeições e calcular os valores nutricionais. Permitir ao paciente consultar seu plano e itens.
RF14	Agendamento de consultas	Permitir ao nutricionista agendar, consultar, filtrar, confirmar, remarcar, cancelar e concluir consultas de pacientes vinculados; apresentar os compromissos ao paciente e impedir sobreposição de horários.
RF15	Gestão de desafios	Permitir ao nutricionista criar, consultar, editar e encerrar desafios, definir categoria, período e metas e incluir participantes.
RF16	Participação em desafios	Permitir ao paciente consultar desafios do nutricionista vinculado, inscrever-se, consultar e registrar check-ins com valores e observações, acompanhar o progresso e concluir sua participação.
RF17	Conversas e mensagens de texto	Permitir a paciente e nutricionista vinculados consultar conversas, listar e paginar o histórico, enviar mensagens de texto, ver mensagens não lidas e registrar a leitura.
RF18	Mensagens de voz	Permitir enviar e reproduzir mensagens de áudio nas conversas autorizadas e disponibilizar a transcrição textual quando informada.
RF19	Resumo administrativo	Apresentar ao administrador indicadores gerais de usuários por perfil e status, alimentos, registros alimentares, planos ativos e calorias médias do catálogo.
RF20	Verificação de serviço	Disponibilizar o endpoint de saúde da API para informar o ambiente e verificar a conexão com o banco de dados.

2.1 REQUISITOS NÃO FUNCIONAIS (RNFs)

ID	Nome do Requisito	Descrição
RNF01	Usabilidade por perfil	Organizar páginas, painéis e funcionalidades conforme os perfis de paciente, nutricionista e administrador.
RNF02	Adaptação da interface	Adaptar a interface a diferentes larguras de tela e ponteiros táteis; respeitar a preferência do navegador por redução de movimento.
RNF03	Compatibilidade web	Utilizar HTML, CSS e JavaScript no frontend web, sem exigir aplicativo móvel nativo.
RNF04	Localização	Apresentar textos, mensagens, datas e horários no idioma e nos formatos brasileiros utilizados pela interface.
RNF05	Manutenibilidade	Separar a implementação em rotas, controllers, serviços, repositórios e infraestrutura, permitindo evolução e testes por camada.
RNF06	Persistência relacional	Utilizar Prisma com banco relacional: SQLite no desenvolvimento local e PostgreSQL na produção.
RNF07	Autenticação segura	Integrar a autenticação ao Supabase Auth, exigir e-mail confirmado e validar o token e a identidade no backend nas rotas protegidas.
RNF08	Controle de acesso	Verificar no backend o perfil e o estado atual da conta, a aprovação profissional e os vínculos de responsabilidade antes de permitir acesso aos dados.
RNF09	Proteção de transporte e origens	Exigir HTTPS para URLs e origens na produção e restringir requisições de outras origens à lista CORS configurada.

Requisitos não funcionais — continuação.

ID	Nome do Requisito	Descrição
RNF10	Proteção de dados e segredos	Manter chaves privadas de integração somente no backend e restringir dados pessoais, de saúde, fotos e áudios aos usuários autorizados; respostas de áudio não devem ser armazenadas em cache.
RNF11	Validação de entradas	Validar formato, tipo, faixa e tamanho dos dados recebidos antes de gravá-los, incluindo campos permitidos, datas, medidas, contatos e nutrientes.
RNF12	Limites de requisição e mídia	Limitar JSON a 1 MB, fotos a 5 MB e áudios a 512 KB e 120 segundos; aceitar somente formatos de imagem e áudio previstos e limitar mensagens de texto a 2.000 caracteres.
RNF13	Integridade dos dados	Usar chaves únicas e relações do banco para manter a consistência dos registros e impedir conflitos de consultas, inclusive por restrição de exclusão no PostgreSQL.
RNF14	Paginação de conversas	Paginar o histórico de mensagens e limitar cada consulta a no máximo 100 mensagens.
RNF15	Tratamento de falhas	Responder erros da API com códigos HTTP e mensagens controladas, não expor detalhes internos ao cliente e registrar falhas inesperadas no servidor.
RNF16	Configuração de produção	Exigir Node.js 20 ou superior, PostgreSQL e credenciais necessárias do Supabase na produção; validar configurações de URL, HTTPS e armazenamento ao iniciar.
RNF17	Armazenamento de fotos	Permitir armazenamento local durante o desenvolvimento e armazenamento Supabase ou desativado na produção; não depender de disco local não persistente em produção.
RNF18	Verificabilidade	Disponibilizar uma suíte automatizada de testes executável pelo comando npm test para verificar funcionalidades, rotas e regras de acesso.

3. PROJETO E ARQUITETURA DO SOFTWARE

3.1 ARQUITETURA GERAL

O NutriFlow utiliza uma arquitetura web em camadas: interface no navegador, API HTTP/REST em Express, lógica organizada em serviços e repositórios, e persistência relacional acessada por Prisma. O Supabase Auth provê autenticação externa.

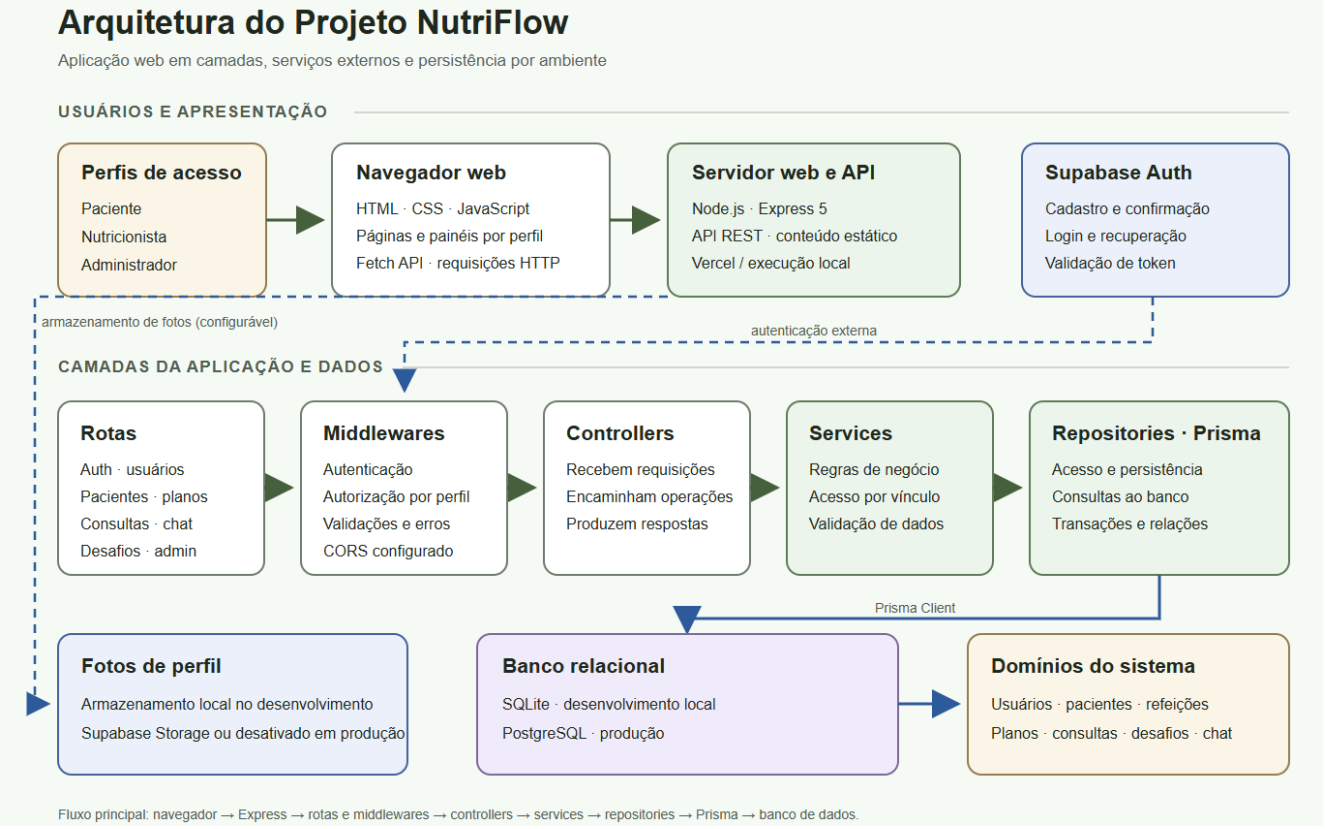
A interface web estática é servida pelo próprio backend Express. A API recebe as requisições, aplica autenticação e autorização, executa a lógica em serviços e repositórios e acessa SQLite ou PostgreSQL por meio do Prisma.

Camada de Apresentação (Frontend): Páginas HTML, estilos CSS e scripts JavaScript executados no navegador; o servidor também disponibiliza os arquivos estáticos.

Camada de Dados (Banco de Dados): API REST em Node.js e Express 5, com controllers, services, repositories, middlewares e validações.

Dados e serviços externos: Prisma ORM com SQLite em desenvolvimento e PostgreSQL em produção; Supabase Auth para autenticação e opção de Supabase Storage para fotos.

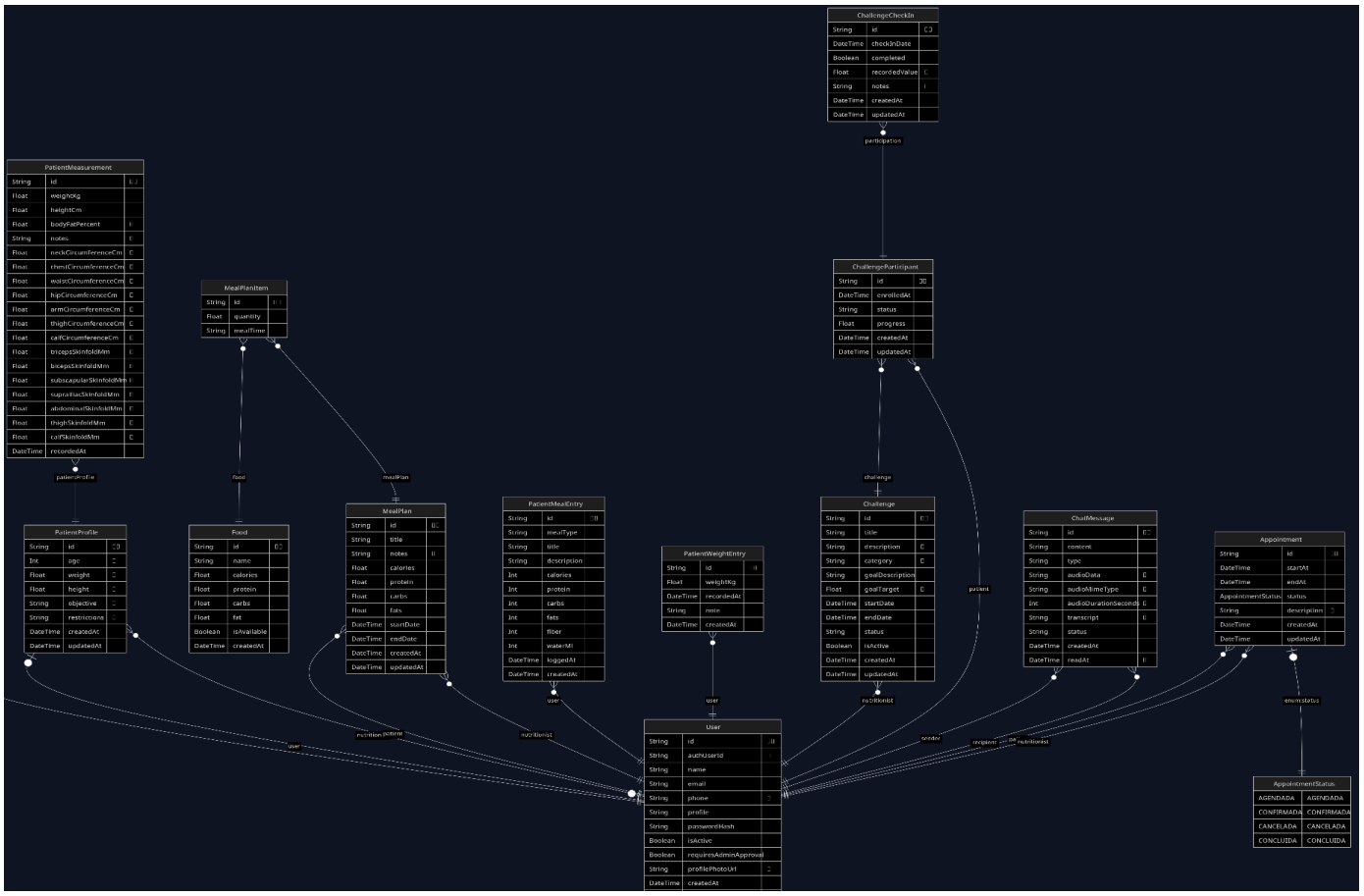
Fluxo: navegador (HTML/CSS/JavaScript) -> API Express -> serviços/repositórios -> Prisma -> SQLite ou PostgreSQL. Supabase Auth é integrado ao fluxo de autenticação.



3.2 PROJETO DO BANCO DE DADOS

O banco de dados é relacional. O esquema Prisma define usuários e perfis, vínculos entre pacientes e nutricionistas, planos alimentares, alimentos, registros de evolução, desafios, mensagens e consultas.

Entidades principais: User, PatientProfile, PatientMeasurement, Food, MealPlan, MealPlanItem, PatientMealEntry, PatientWeightEntry, Challenge, ChallengeParticipant, ChallengeCheckIn, ChatMessage, Appointment e NutritionistApprovalRequest.



Dicionário de dados

1. Usuários

Nome do campo	Tipo de dados	Chave (PK/FK)	Nulo?	Descrição
id	String (UUID)	PK	Não	Identificador único do usuário.
authUserId	String	Único	Sim	Identificador correspondente do usuário no Supabase Auth.
name	String	—	Não	Nome do usuário.
email	String	Único	Não	Endereço de e-mail do usuário.
phone	String	—	Sim	Telefone de contato do usuário.
profile	String	—	Não	Perfil ou tipo de acesso do usuário no sistema.
passwordHash	String	—	Não	Hash da senha armazenada pelo sistema.
isActive	Boolean	—	Não	Indica se a conta está ativa. Padrão: true.
requiresAdminApproval	Boolean	—	Não	Indica se o usuário precisa de aprovação administrativa. Padrão: false.
profilePhotoUrl	String	—	Sim	URL da foto de perfil do usuário.
createdAt	DateTime	—	Não	Data e hora de criação do cadastro. Padrão: data atual.
updatedAt	DateTime	—	Não	Data e hora da última atualização do cadastro.

2. Solicitações de aprovação

Nome do campo	Tipo de dados	Chave (PK/FK)	Nulo?	Descrição
id	String (UUID)	PK	Não	Identificador único da solicitação.
userId	String	FK / Único	Não	Referência ao usuário que solicitou a aprovação.
status	String	—	Não	Situação da solicitação. Padrão: PENDING.
decisionReason	String	—	Sim	Motivo informado para a decisão da solicitação.
createdAt	DateTime	—	Não	Data e hora de criação da solicitação.
updatedAt	DateTime	—	Não	Data e hora da última atualização da solicitação.

3. Perfil do paciente

Nome do campo	Tipo de dados	Chave (PK/FK)	Nulo?	Descrição
id	String (UUID)	PK	Não	Identificador único do perfil do paciente.
userId	String	FK / Único	Não	Referência ao usuário proprietário do perfil.
age	Int	—	Sim	Idade informada pelo paciente.
weight	Float	—	Sim	Peso informado no perfil.
height	Float	—	Sim	Altura informada no perfil.
objective	String	—	Sim	Objetivo nutricional informado pelo paciente.
restrictions	String	—	Sim	Restrições alimentares informadas pelo paciente.
createdAt	DateTime	—	Não	Data e hora de criação do perfil.
updatedAt	DateTime	—	Não	Data e hora da última atualização do perfil.
nutritionistId	String	FK	Sim	Referência ao nutricionista associado ao paciente.

4. Medições do paciente

Nome do campo	Tipo de dados	Chave (PK/FK)	Nulo?	Descrição
id	String (UUID)	PK	Não	Identificador único da medição.
patientProfileId	String	FK	Não	Referência ao perfil do paciente relacionado à medição.
weightKg	Float	—	Não	Peso corporal em quilogramas.
heightCm	Float	—	Não	Altura em centímetros.
bodyFatPercent	Float	—	Sim	Percentual de gordura corporal.
notes	String	—	Sim	Observações sobre a medição.
neckCircumferenceCm	Float	—	Sim	Circunferência do pescoço em centímetros.
chestCircumferenceCm	Float	—	Sim	Circunferência do tórax em centímetros.
waistCircumferenceCm	Float	—	Sim	Circunferência da cintura em centímetros.
hipCircumferenceCm	Float	—	Sim	Circunferência do quadril em centímetros.
armCircumferenceCm	Float	—	Sim	Circunferência do braço em centímetros.
thighCircumferenceCm	Float	—	Sim	Circunferência da coxa em centímetros.
calfCircumferenceCm	Float	—	Sim	Circunferência da panturrilha em centímetros.
tricepsSkinfoldMm	Float	—	Sim	Dobra cutânea do tríceps em milímetros.
bicepsSkinfoldMm	Float	—	Sim	Dobra cutânea do bíceps em milímetros.
subscapularSkinfoldMm	Float	—	Sim	Dobra cutânea subescapular em milímetros.
suprailiacSkinfoldMm	Float	—	Sim	Dobra cutânea supra-ilíaca em milímetros.
abdominalSkinfoldMm	Float	—	Sim	Dobra cutânea abdominal em milímetros.
thighSkinfoldMm	Float	—	Sim	Dobra cutânea da coxa em milímetros.
calfSkinfoldMm	Float	—	Sim	Dobra cutânea da panturrilha em milímetros.
recordedAt	DateTime	—	Não	Data e hora em que a medição foi registrada. Padrão: data atual.

5. Food — Alimentos

.

Nome do campo	Tipo de dados	Chave (PK/FK)	Nulo?	Descrição
id	String	PK	Não	Identificador único do alimento.
name	String	Único	Não	Nome do alimento.
calories	Float	—	Não	Quantidade de calorias associada ao alimento.
protein	Float	—	Não	Quantidade de proteínas.
carbs	Float	—	Não	Quantidade de carboidratos.
fat	Float	—	Não	Quantidade de gorduras.
isAvailable	Boolean	—	Não	Indica se o alimento está disponível no catálogo. Padrão: true.
createdAt	DateTime	—	Não	Data e hora de inclusão do alimento. Padrão: data atual.

6. Planos alimentares

Nome do campo	Tipo de dados	Chave (PK/FK)	Nulo?	Descrição
id	String (UUID)	PK	Não	Identificador único do plano alimentar.
patientId	String	FK	Não	Referência ao paciente que receberá o plano.
nutritionistId	String	FK	Não	Referência ao nutricionista responsável pelo plano.
title	String	—	Não	Título do plano alimentar.
notes	String	—	Sim	Observações adicionais sobre o plano.
calories	Float	—	Não	Total de calorias do plano. Padrão: 0.
protein	Float	—	Não	Total de proteínas do plano. Padrão: 0.
carbs	Float	—	Não	Total de carboidratos do plano. Padrão: 0.
fats	Float	—	Não	Total de gorduras do plano. Padrão: 0.
startDate	DateTime	—	Não	Data de início da validade do plano. Padrão: data atual.
endDate	DateTime	—	Não	Data de encerramento da validade do plano.
createdAt	DateTime	—	Não	Data e hora de criação do plano. Padrão: data atual.
updatedAt	DateTime	—	Não	Data e hora da última atualização do plano.

7. Itens dos planos alimentares

Nome do campo	Tipo de dados	Chave (PK/FK)	Nulo?	Descrição
id	String (UUID)	PK	Não	Identificador único do item do plano.
mealPlanId	String	FK	Não	Referência ao plano alimentar ao qual o item pertence.
foodId	String	FK	Não	Referência ao alimento incluído no plano.
quantity	Float	—	Não	Quantidade do alimento indicada no plano.
mealTime	String	—	Não	Horário ou identificação da refeição em que o alimento será consumido.

8. Registros alimentares

Nome do campo	Tipo de dados	Chave (PK/FK)	Nulo?	Descrição
id	String (UUID)	PK	Não	Identificador único do registro alimentar.
userId	String	FK	Não	Referência ao usuário que realizou o registro.
mealType	String	—	Não	Tipo de refeição registrada.
title	String	—	Não	Título ou nome da refeição.
description	String	—	Não	Descrição da refeição. Padrão: texto vazio.
calories	Int	—	Não	Quantidade de calorias da refeição.
protein	Int	—	Não	Quantidade de proteínas da refeição.
carbs	Int	—	Não	Quantidade de carboidratos da refeição.
fats	Int	—	Não	Quantidade de gorduras da refeição.
fiber	Int	—	Não	Quantidade de fibras da refeição.
waterMl	Int	—	Não	Quantidade de água registrada em mililitros.
loggedAt	DateTime	—	Não	Data e hora em que a refeição foi registrada ou consumida.
createdAt	DateTime	—	Não	Data e hora de criação do registro. Padrão: data atual.

9. Registros de peso

Nome do campo	Tipo de dados	Chave (PK/FK)	Nulo?	Descrição
id	String (UUID)	PK	Não	Identificador único do registro de peso.
userId	String	FK	Não	Referência ao usuário proprietário do registro.
weightKg	Float	—	Não	Peso registrado em quilogramas.
recordedAt	DateTime	—	Não	Data e hora da medição do peso.
note	String	—	Não	Observação sobre o registro. Padrão: texto vazio.
createdAt	DateTime	—	Não	Data e hora de criação do registro. Padrão: data atual.

10. Desafios nutricionais

Nome do campo	Tipo de dados	Chave (PK/FK)	Nulo?	Descrição
id	String (UUID)	PK	Não	Identificador único do desafio.
nutritionistId	String	FK	Não	Referência ao nutricionista responsável pela criação do desafio.
title	String	—	Não	Título do desafio nutricional.
description	String	—	Sim	Descrição e orientações do desafio.
category	String	—	Sim	Categoria do desafio.
goalDescription	String	—	Não	Descrição da meta que deverá ser alcançada.
goalTarget	Float	—	Sim	Valor numérico definido como objetivo do desafio.
startDate	DateTime	—	Não	Data e hora de início do desafio.
endDate	DateTime	—	Não	Data e hora de encerramento do desafio.
status	String	—	Não	Situação atual do desafio. Padrão: AGENDADO.
isActive	Boolean	—	Não	Indica se o desafio está ativo. Padrão: true.
createdAt	DateTime	—	Não	Data e hora de criação do desafio. Padrão: data atual.
updatedAt	DateTime	—	Não	Data e hora da última atualização do desafio.

11. Participantes dos desafios

Nome do campo	Tipo de dados	Chave (PK/FK)	Nulo?	Descrição
id	String (UUID)	PK	Não	Identificador único da participação.
challengeId	String	FK	Não	Referência ao desafio no qual o paciente está inscrito.
patientId	String	FK	Não	Referência ao usuário participante do desafio.
enrolledAt	DateTime	—	Não	Data e hora da inscrição do paciente. Padrão: data atual.
status	String	—	Não	Situação da participação. Padrão: INSCRITO.
progress	Float	—	Não	Percentual ou valor de progresso registrado para a participação. Padrão: 0.
createdAt	DateTime	—	Não	Data e hora de criação do registro de participação. Padrão: data atual.
updatedAt	DateTime	—	Não	Data e hora da última atualização da participação.

12. Check-ins dos desafios

Nome do campo	Tipo de dados	Chave (PK/FK)	Nulo?	Descrição
id	String (UUID)	PK	Não	Identificador único do check-in.
participationId	String	FK composta	Não	Referência à participação do paciente no desafio.
patientId	String	FK composta	Não	Identificador do paciente associado à participação.
checkInDate	DateTime (DATE)	—	Não	Data do check-in, armazenada sem informação de horário.
completed	Boolean	—	Não	Indica se a atividade ou meta do dia foi concluída. Padrão: false.
recordedValue	Float	—	Sim	Valor numérico registrado pelo paciente no check-in.
notes	String	—	Sim	Observações adicionadas ao check-in.
createdAt	DateTime	—	Não	Data e hora de criação do check-in. Padrão: data atual.
updatedAt	DateTime	—	Não	Data e hora da última atualização do check-in.

13. Mensagens do chat

Nome do campo	Tipo de dados	Chave (PK/FK)	Nulo?	Descrição
id	String (UUID)	PK	Não	Identificador único da mensagem.
senderId	String	FK	Não	Referência ao usuário que enviou a mensagem.
recipientId	String	FK	Não	Referência ao usuário que recebeu a mensagem.
content	String	—	Não	Conteúdo textual da mensagem.
type	String	—	Não	Tipo da mensagem. Padrão: TEXT.
audioData	String	—	Sim	Dados do áudio da mensagem.
audioMimeType	String	—	Sim	Tipo MIME do arquivo de áudio.
audioDurationSeconds	Int	—	Sim	Duração do áudio em segundos.
transcript	String	—	Sim	Transcrição textual do áudio.
status	String	—	Não	Situação de leitura da mensagem. Padrão: UNREAD.
createdAt	DateTime	—	Não	Data e hora de envio ou criação da mensagem. Padrão: data atual.
readAt	DateTime	—	Sim	Data e hora em que a mensagem foi lida.

14. Agendamentos de consultas

Nome do campo	Tipo de dados	Chave (PK/FK)	Nulo?	Descrição
id	String (UUID)	PK	Não	Identificador único do agendamento.
patientId	String	FK	Não	Referência ao paciente da consulta.
nutritionistId	String	FK	Não	Referência ao nutricionista responsável pela consulta.
startAt	DateTime	—	Não	Data e hora de início da consulta.
endAt	DateTime	—	Não	Data e hora de término da consulta.
status	AppointmentStatus (Enum)	—	Não	Situação do agendamento. Padrão: AGENDADA.
description	String	—	Sim	Descrição ou observações sobre a consulta.
createdAt	DateTime	—	Não	Data e hora de criação do agendamento. Padrão: data atual.
updatedAt	DateTime	—	Não	Data e hora da última atualização do agendamento.

15. Situações dos agendamentos

Nome do valor	Tipo de dados	Chave (PK/FK)	Nulo?	Descrição
AGENDADA	AppointmentStatus	—	—	Consulta criada e aguardando confirmação ou realização.
CONFIRMADA	AppointmentStatus	—	—	Consulta confirmada.
CANCELADA	AppointmentStatus	—	—	Consulta cancelada.
CONCLUIDA	AppointmentStatus	—	—	Consulta concluída.

3.3 PROJETO DE API

A API é REST e está agrupada por domínio. A autenticação é integrada ao Supabase; o backend aplica autorização por perfil e valida os vínculos necessários. O repositório não contém especificação OpenAPI nem interface Swagger.

Principais grupos de rotas: /api/auth (conta e autenticação), /api/patients e /api/patient (perfil e acompanhamento), /api/nutritionist (gestão profissional), /api/appointments (consultas),
/api/chat e /chat (mensagens) e /api/admin (administração).

As rotas aplicam autenticação e autorização no backend. O endpoint /health informa a disponibilidade da aplicação e a conexão com o banco de dados.

Documentação da API:

Documentação OpenAPI/Swagger: não publicada. Código das rotas: diretório src/routes/ no repositório NutriFlow.

Endpoints de referência incluem cadastro e login em /api/auth, registro de medidas em
/api/patients/:id/measurements e administração de usuários em /api/admin/users.

API REST do NutriFlow: endpoints agrupados por domínio e protegidos conforme o perfil de acesso.

Captura da interface Swagger não disponível: o projeto não inclui Swagger UI.

Para consulta offline, as definições das rotas e seus controllers estão no código-fonte, principalmente em src/routes/ e src/controllers/.

4. TECNOLOGIAS E FERRAMENTAS

4.1 STACK DE TECNOLOGIAS

A pilha tecnológica foi selecionada para manter a aplicação web simples de executar e permitir a separação entre interface, regras de negócio e acesso a dados.

Frontend: **HTML, CSS e JavaScrip**t sem framework de interface; essa escolha mantém a aplicação leve e é compatível com a estrutura existente.

HTML5 e JavaScript: estruturam as páginas e interações servidas pelo

Express. CSS: define o layout e a apresentação responsiva da interface.

Fetch API: realiza chamadas HTTP do navegador para os endpoints do

backend. Não são usados React, Vite, Axios ou Tailwind CSS no frontend

atual.

Backend e dados:

Node.js 20 ou superior: ambiente de execução do servidor e da API.

Express 5: framework HTTP para API, middlewares e entrega dos arquivos da interface.

Prisma ORM 6: mapeia o schema relacional e oferece acesso consistente a SQLite e PostgreSQL.

Supabase Auth: provê autenticação; o backend valida tokens e aplica regras de autorização por perfil.

4.2 FERRAMENTAS DE DESENVOLVIMENTO

As ferramentas identificadas no repositório e no fluxo de desenvolvimento apoiam controle de versão, execução local, persistência e testes automatizados.

IDE: Vscode pode ser utilizado para editar e executar o projeto.

Controle de versão: Git; repositório remoto hospedado no GitHub em

PA2-2026-2/NutriFlow. Testes: Jest e Supertest, executados por npm test.

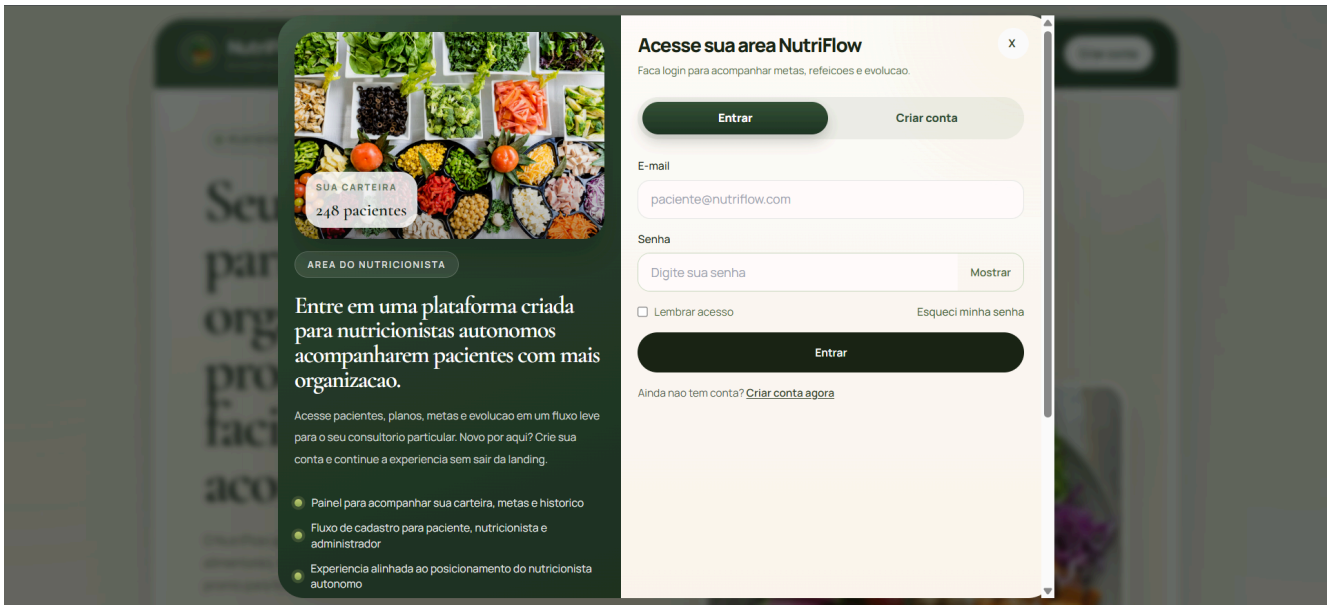
Dados: Prisma CLI para geração de clientes e sincronização do banco local; Supabase para autenticação configurada.

5. IMPLEMENTAÇÃO E RESULTADOS

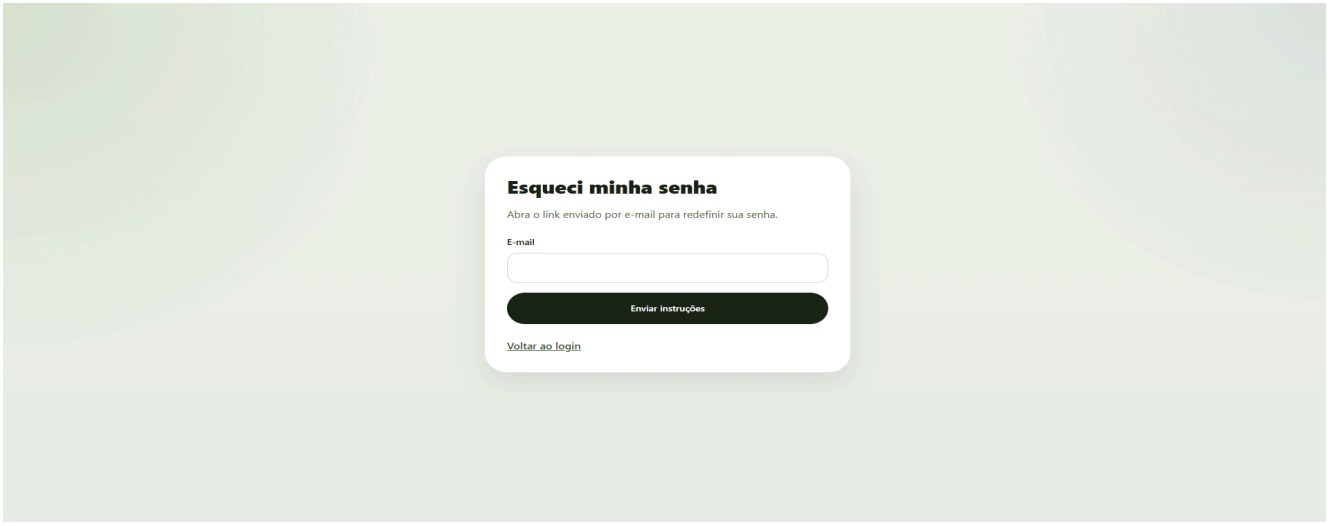
5.1 TELAS DO SISTEMA

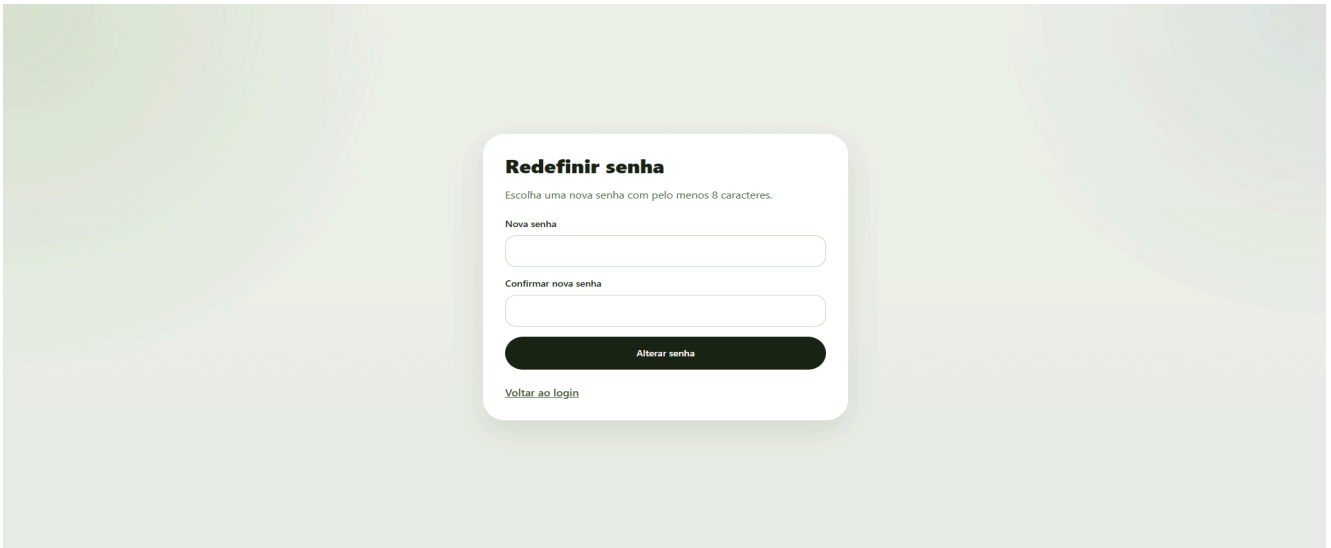
As telas descritas representam as principais áreas do sistema. Não foram localizados arquivos de captura de tela no repositório; as imagens devem ser capturadas na aplicação para uma versão ilustrada do documento.

Tela de Login



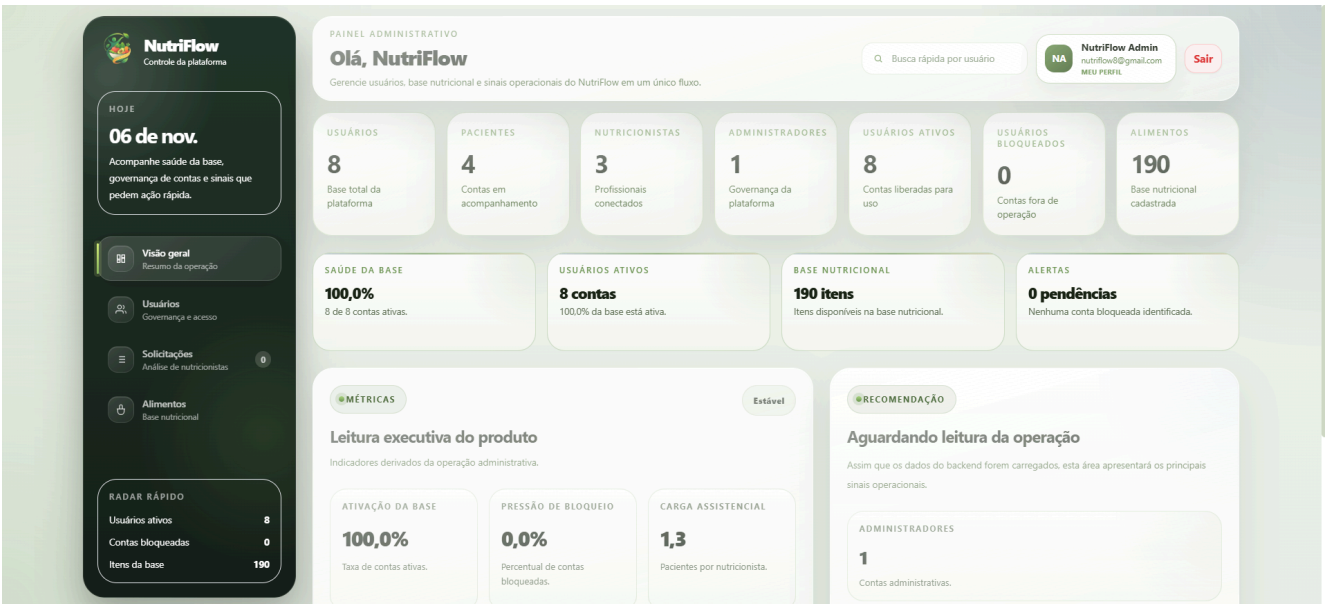
Tela Redefinição de senha:



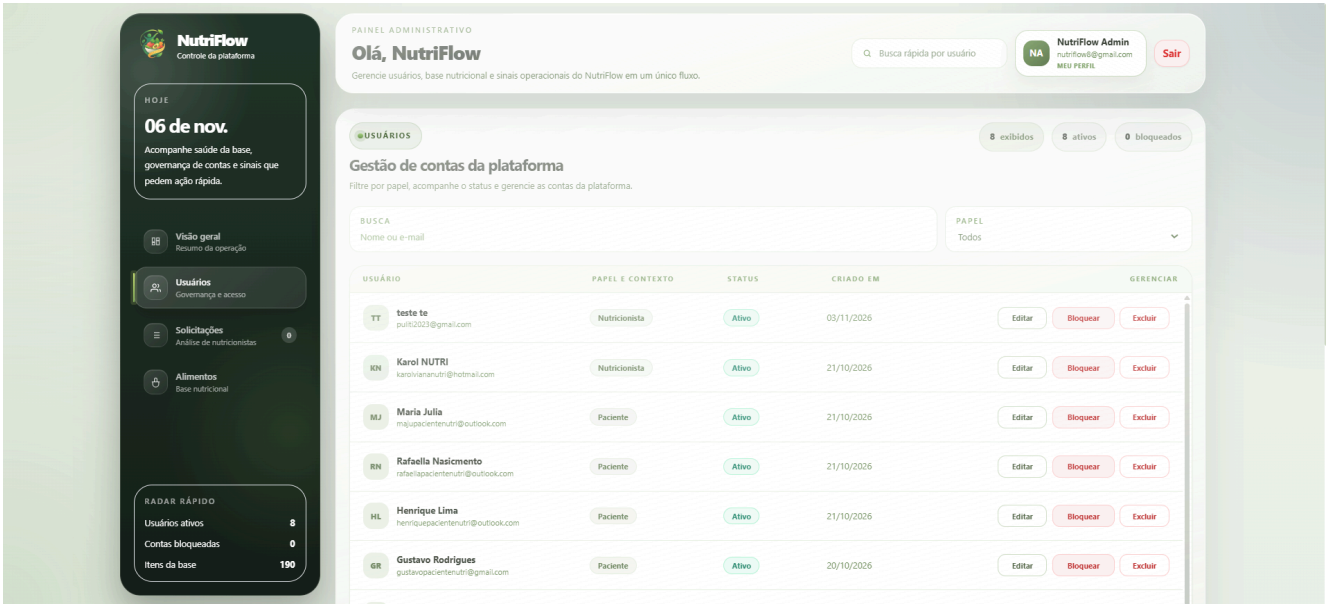


Telas Administrador

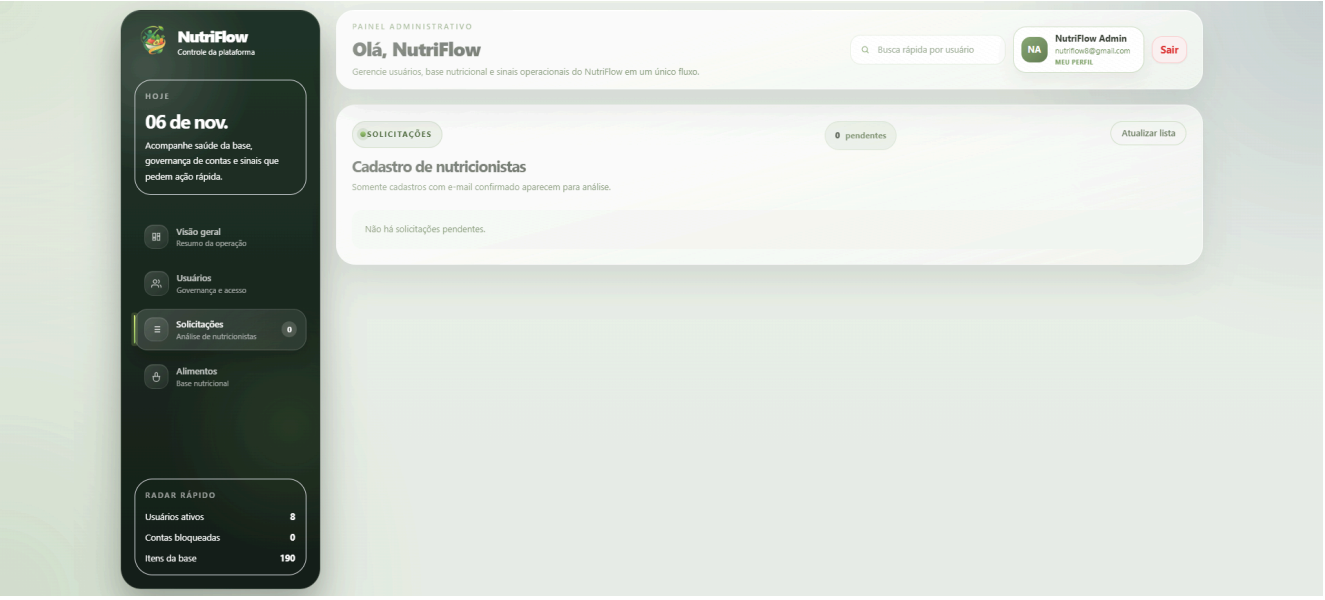
Painel Administrador:



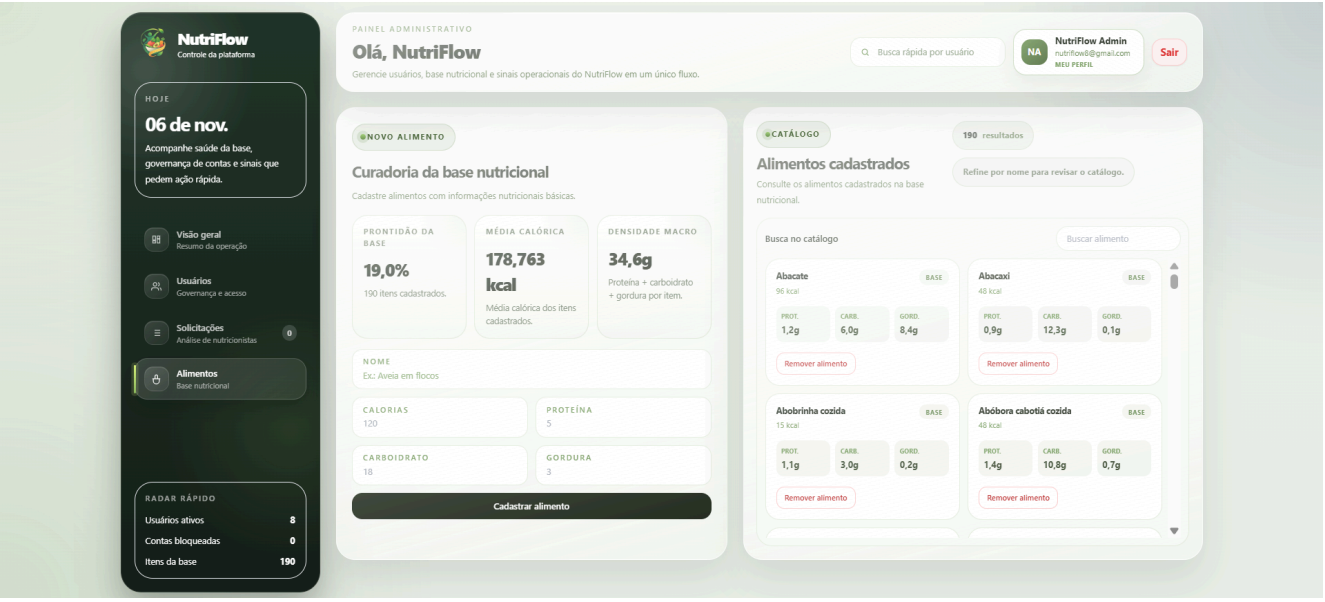
Controle de Usuários:



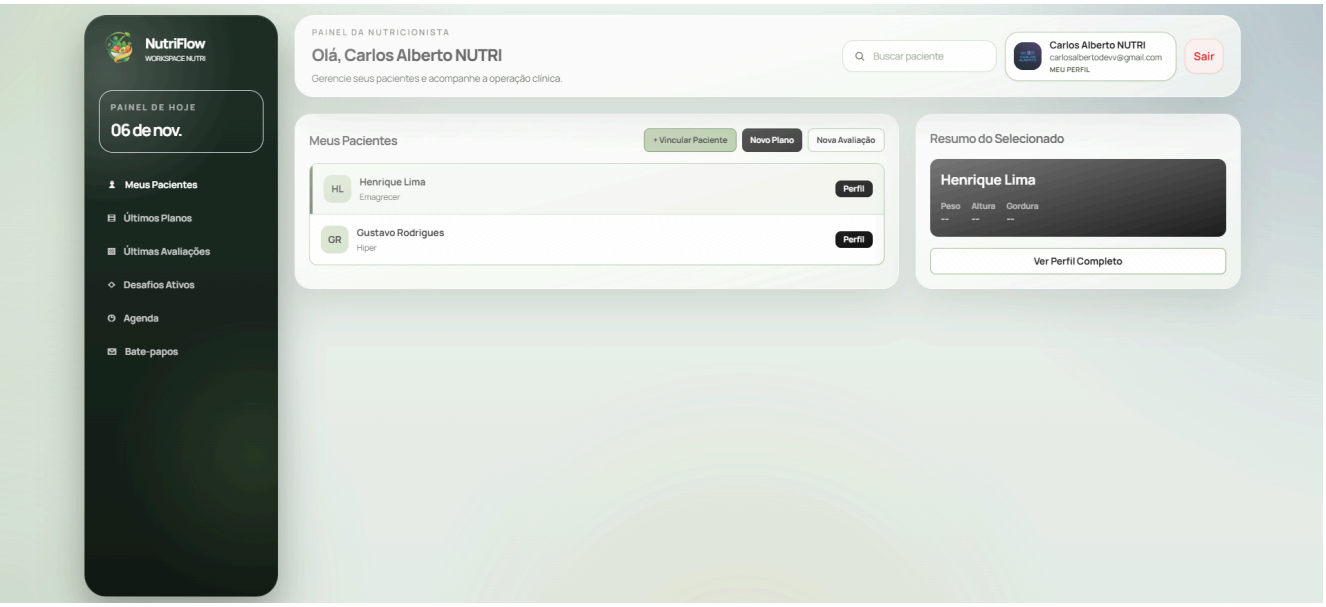
Tela Solicitações de Nutricionistas:



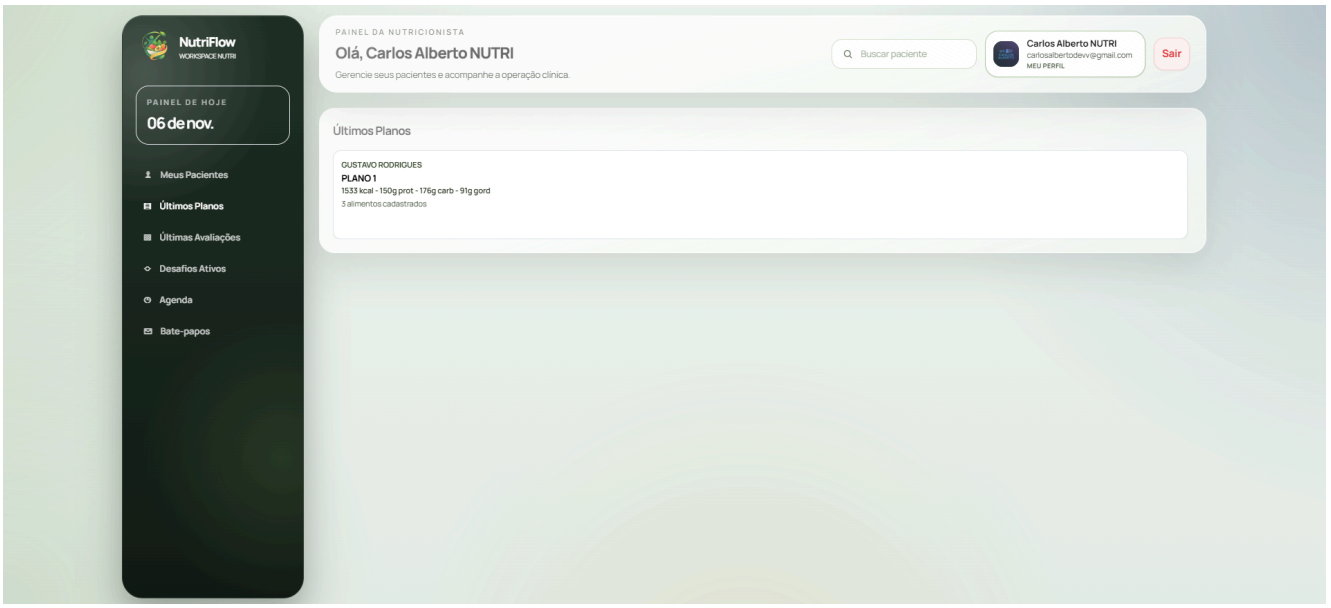
Tela Controle de Alimentos:



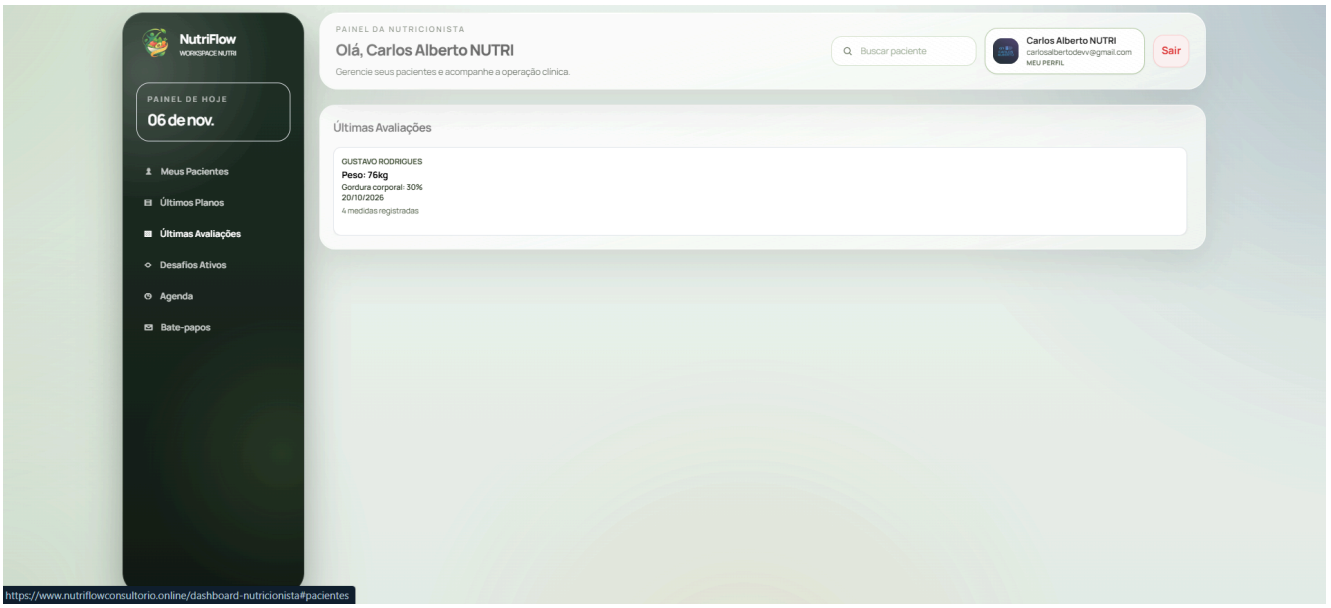
Painel Nutricionista:



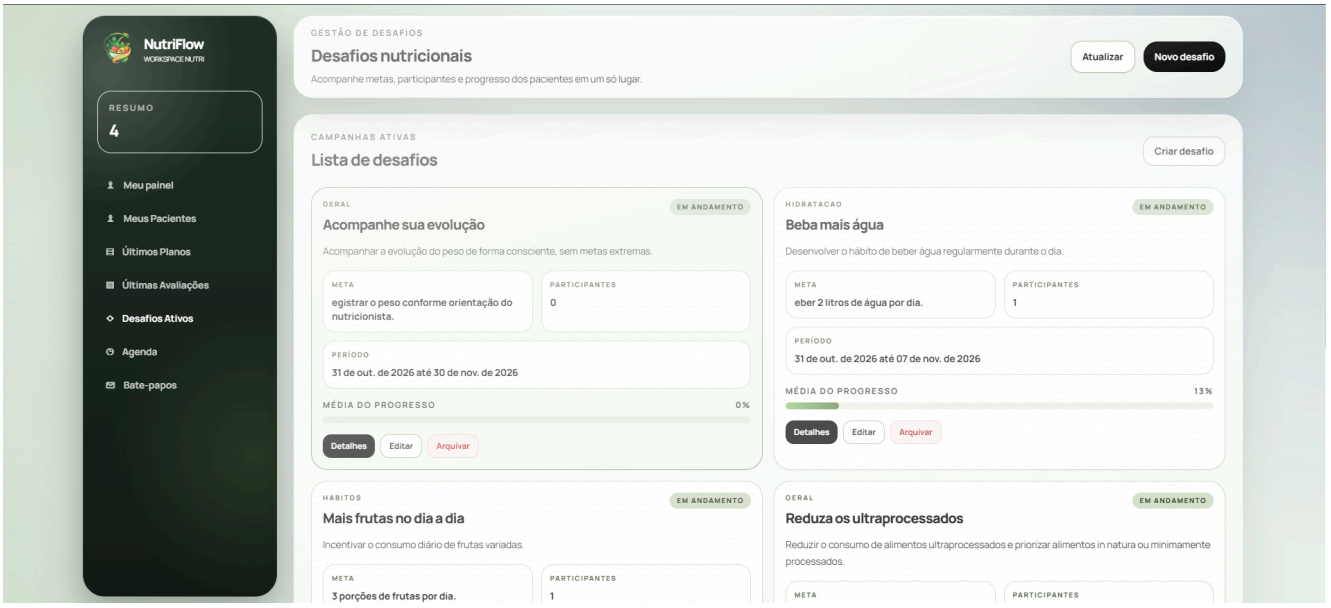
Tela Planos Alimentares ativos:



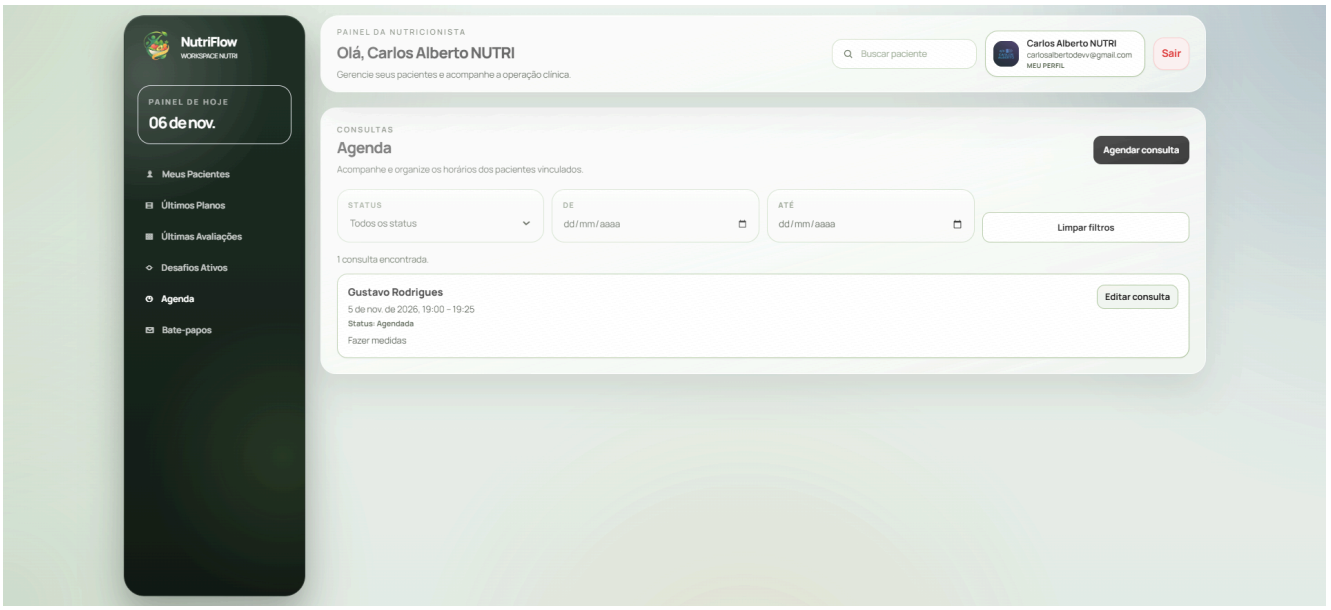
Tela Avaliações realizadas:



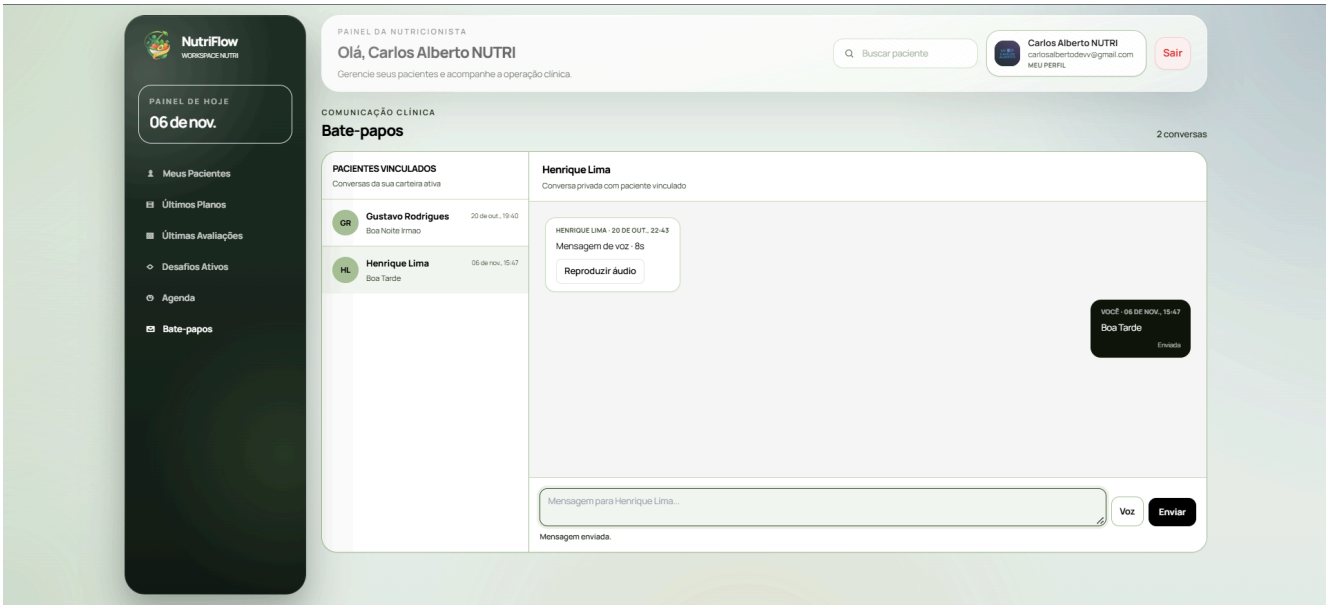
Tela Desafios Criados:



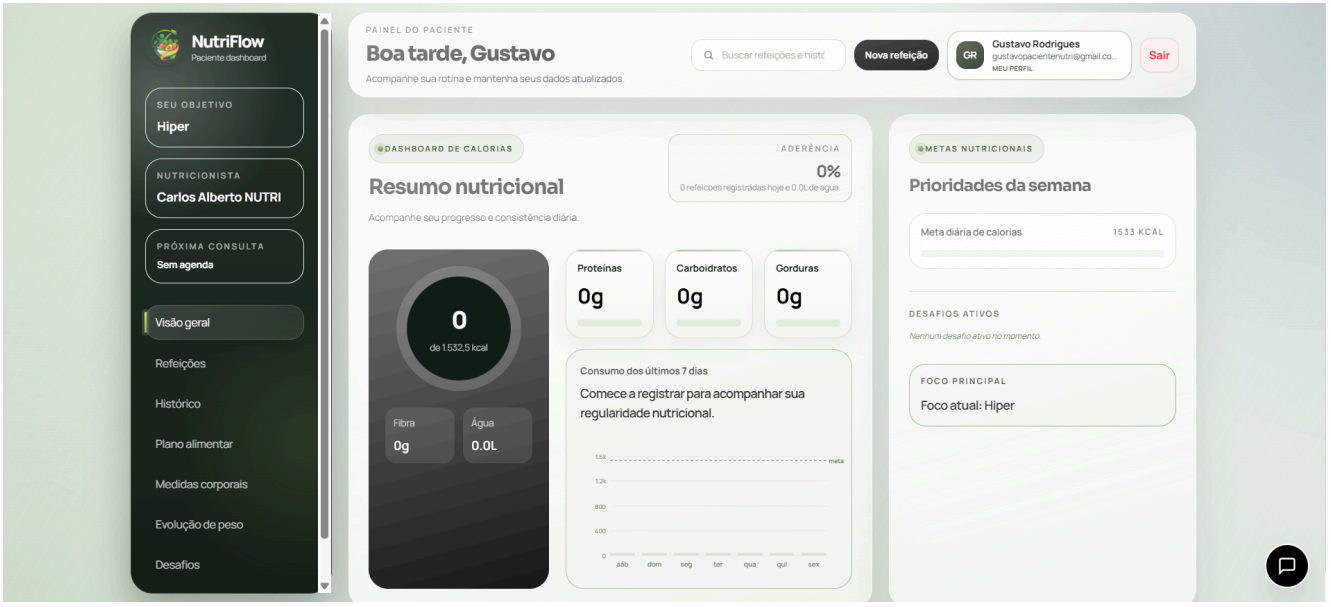
Tela Controle de Agenda:



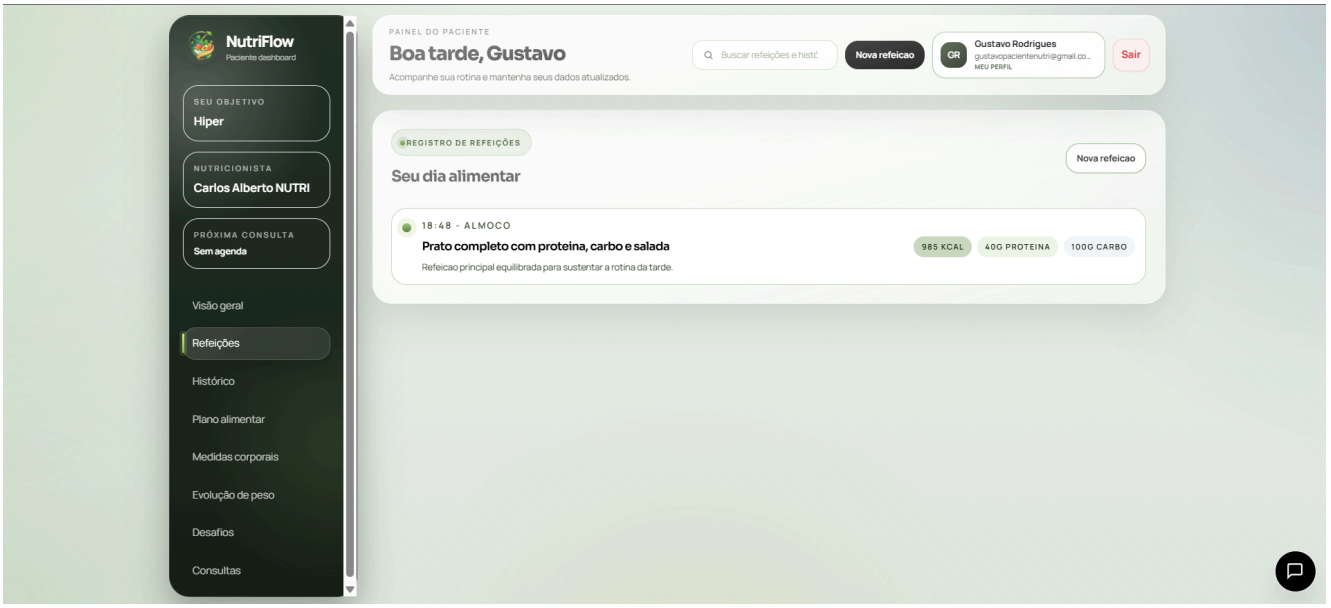
Tela Bate Papo com paciente:



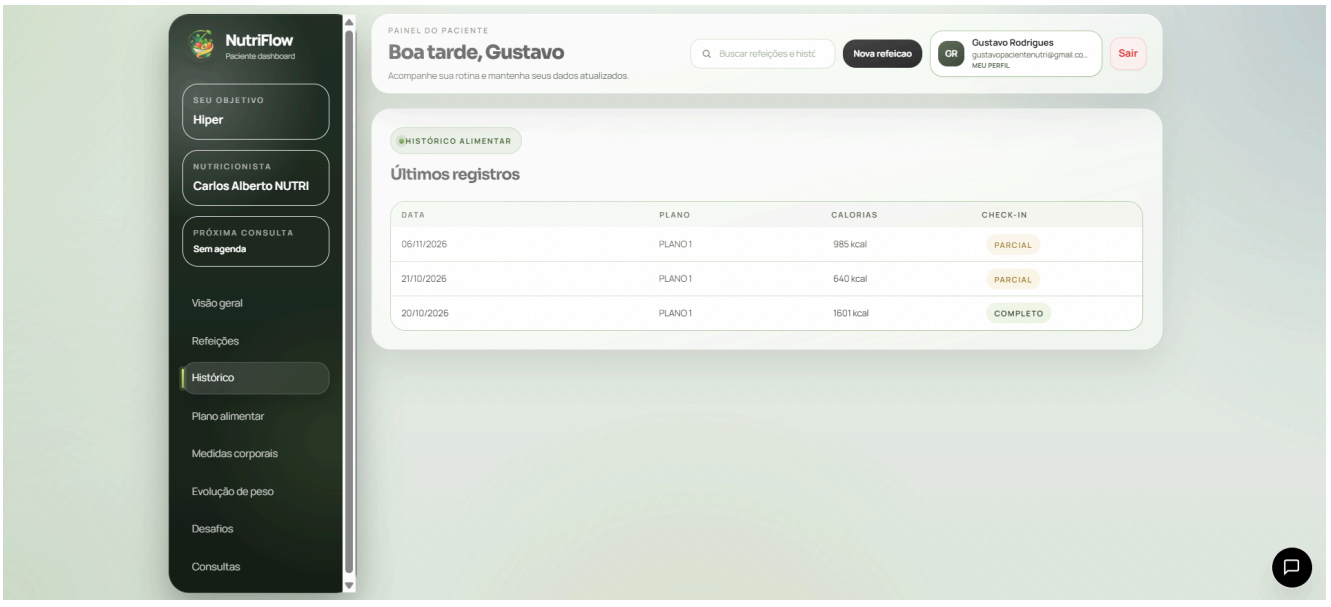
Painel Paciente:



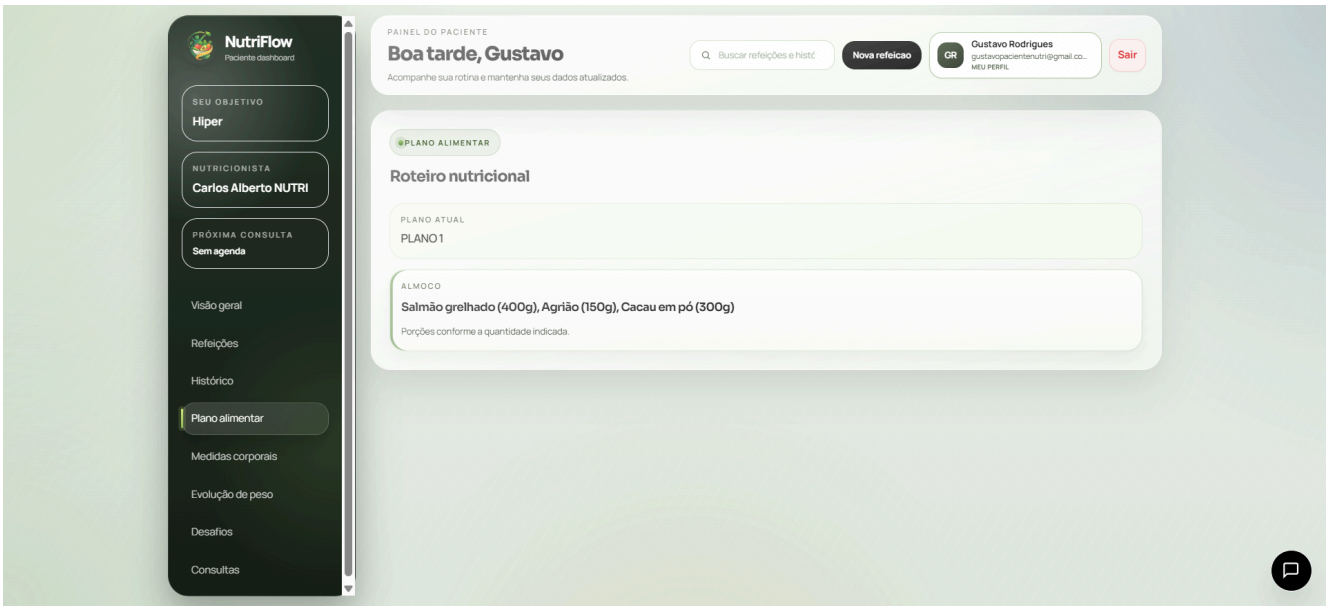
Tela Refeições do dia:



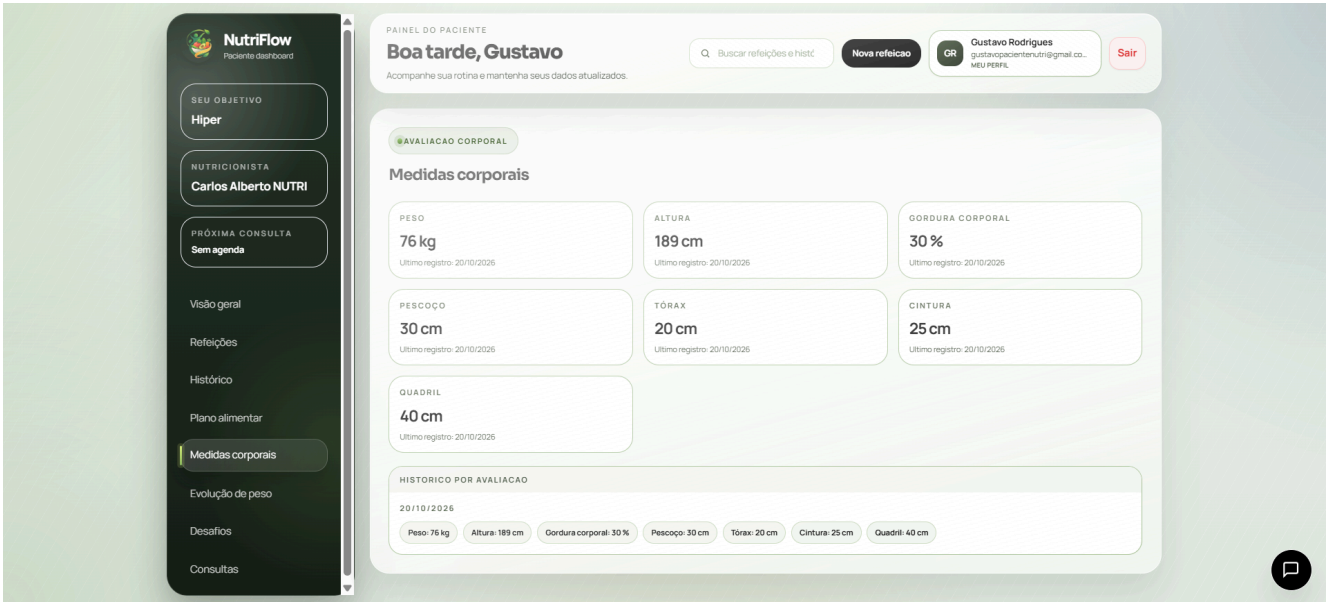
Tela Histórico Alimentar:



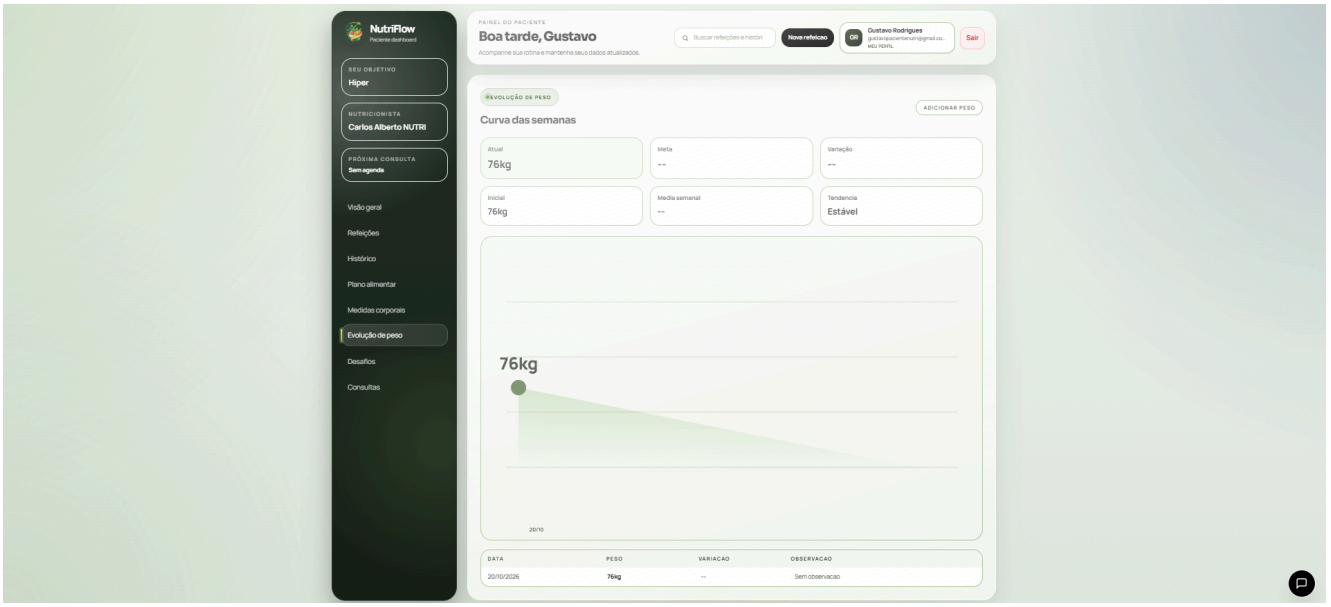
Tela Plano Alimentar(paciente):



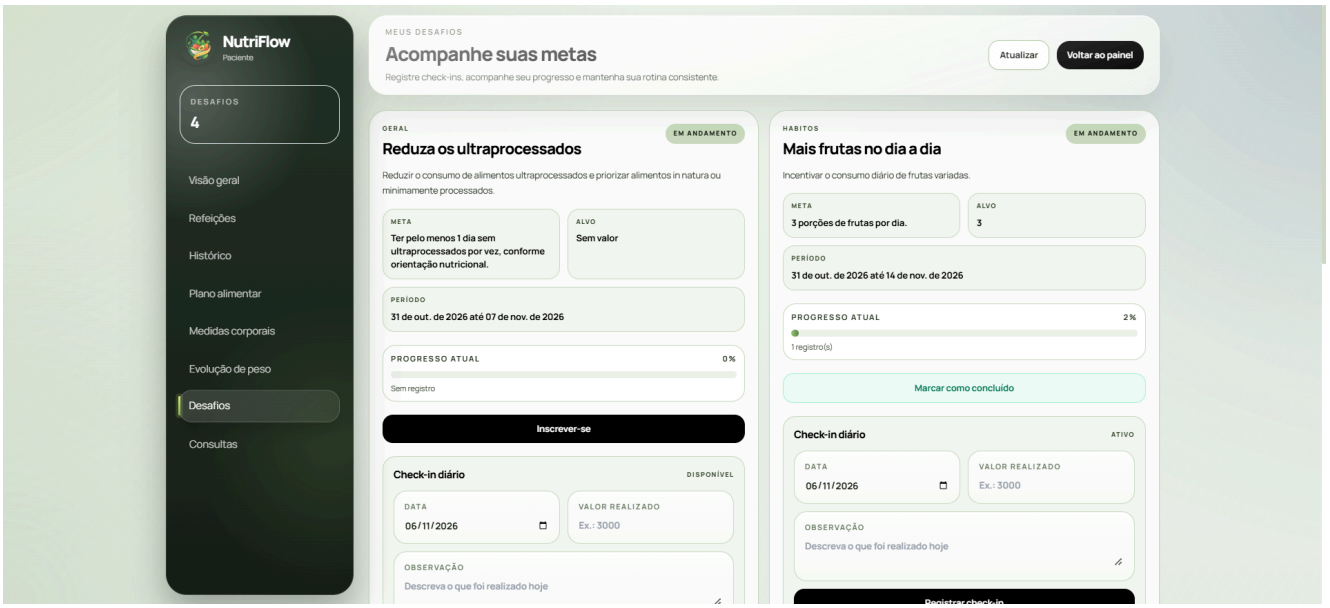
Tela Medidas Corporais:



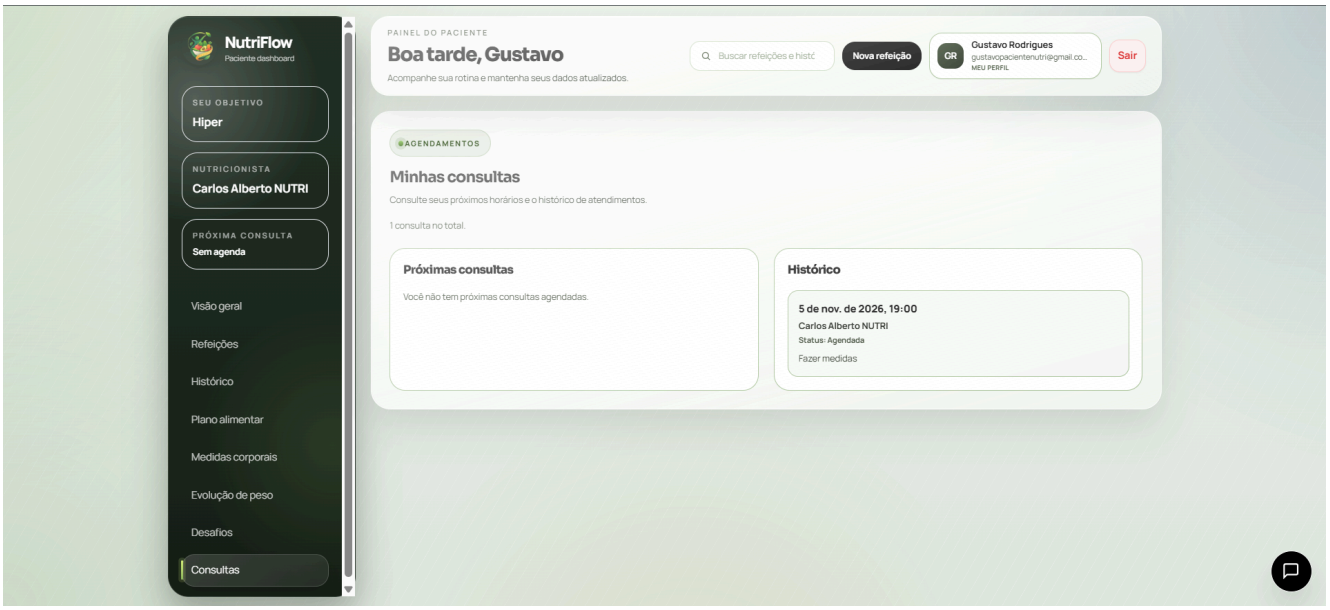
Tela Evolução Peso:



Tela Desafios(Paciente):



Tela Consultas Agendadas(Paciente):



6. AMBIENTE E GUIA DE IMPLANTAÇÃO

A aplicação é executada localmente com um único servidor Node.js que serve a interface estática e a API; não há etapa de build separada do frontend.

6.1 REQUISITOS DO AMBIENTE

Ambiente necessário para desenvolvimento local: sistema operacional compatível com Node.js, Node.js 20 ou superior e npm. SQLite é usado localmente; PostgreSQL é usado na configuração de produção. Supabase é necessário para autenticação real.

Sistema operacional: Windows, macOS ou Linux compatível com Node.js 20 ou superior.

Runtime: Node.js 20 ou superior.

Gerenciador de pacotes: npm, incluído com Node.js.

Banco de dados: SQLite para desenvolvimento; PostgreSQL para produção.

Serviços opcionais: projeto Supabase com Auth para habilitar autenticação; configure as variáveis de ambiente conforme o README.

6.2 PROCESSO DE IMPLANTAÇÃO

Implantação local a partir do terminal, na raiz do repositório:

1. Clone o repositório:

git clone https://github.com/PA2-2026-2/NutriFlow.git

2. Entre na pasta: cd NutriFlow

3. Instale as dependências: npm install

4. Prepare o banco SQLite local: npm run db:push

5. Para autenticação, configure SUPABASE_URL e SUPABASE_ANON_KEY no arquivo .env. Sem Supabase, é possível iniciar o servidor com SQLite, mas os fluxos de autenticação externa não estarão disponíveis.

6. Inicie o servidor: npm run dev

7. Acesse http://127.0.0.1:3000/; a API e a interface são servidas pelo mesmo processo.

Frontend: não requer instalação ou build separado; o Express disponibiliza os arquivos estáticos.

Para executar os testes automatizados: npm test

Verifique o servidor e o banco em http://127.0.0.1:3000/health.

O frontend não requer instalação separada.

6.3 ACESSO À APLICAÇÃO IMPLANTADA

A aplicação publicada informada pela equipe está disponível no endereço abaixo. Não foram fornecidas credenciais de demonstração.

URL da aplicação: https://www.nutriflowconsultorio.online

Credenciais de Acesso

(Perfil de Administrador):

Email: nutriflow8@gmail.com

Senha: nutriflow123

(Perfil de Paciente):

Email: gustavopacientenutri@gmail.com

Email: henriquepacientenutri@outlook.com

Senha: nutriflow123

(Perfil de Nutricionista):

Email: karolviananutri@hotmail.com

Senha: nutriflow123

7. CONCLUSÃO

O NutriFlow reúne recursos para apoiar o acompanhamento nutricional em uma aplicação web com perfis distintos, registro de evolução e comunicação integrada. A arquitetura e os controles de acesso oferecem uma base para evoluir o produto com foco em organização, privacidade e facilidade de uso.

7.1 TRABALHOS FUTUROS

- Como trabalhos futuros, são consideradas melhorias que ampliam o acompanhamento sem alterar o escopo atual do produto.
- Evoluir relatórios e visualizações de progresso para apoiar a análise longitudinal de pacientes.
- Ampliar mecanismos de exportação e integração controlada de dados com serviços externos.
- Adicionar notificações configuráveis e outros recursos de comunicação, com consentimento e controles de privacidade.

7.2 GRUPO DE TRABALHO

Integrantes: **Carlos Alberto Freitas (matrícula 2422750)** e **Andrei Araripe (matrícula 2422750).** Responsabilidades: análise, desenvolvimento, testes e documentação do NutriFlow.

7.3 LIÇÕES APRENDIDAS

O desenvolvimento do NutriFlow evidenciou a importância de delimitar permissões, separar responsabilidades e manter consistência entre ambientes de banco de dados e autenticação.

A organização em rotas, controllers, serviços, repositórios e infraestrutura ajudou a separar responsabilidades. A implementação também reforçou que autenticação e autorização são etapas distintas: além de identificar a conta, cada operação precisa validar o perfil e o vínculo com os dados acessados.

A principal lição é projetar os limites de acesso e os relacionamentos de dados desde o início, testar os fluxos integrados e documentar requisitos e configuração de ambiente para reduzir retrabalho.

